

Proposta de Trabalho

Técnicas de Integração de Sistemas (N703)

API de Agregação de Dados Climáticos e Geográficos

1. Conhecimentos necessários para o desenvolvimento da atividade

Para o desenvolvimento desta atividade, os alunos deverão possuir ou desenvolver os seguintes conhecimentos:

- Fundamentos de programação e desenvolvimento web
- Conceitos básicos de APIs REST e integração de sistemas
- Consumo de APIs públicas externas
- Versionamento de código com Git e GitHub
- Conhecimentos básicos de JSON
- Conhecimentos na linguagem de marcação Markdown

2. Objetivo da proposta de atividade

Esta atividade tem como objetivo desenvolver habilidades práticas de integração de sistemas através da criação de uma **API DE AGREGAÇÃO DE DADOS** que consuma APIs públicas externas e retorne dados combinados/enriquecidos em um formato padronizado.

Os alunos deverão aplicar conceitos de:

- Consumo de APIs externas
- Transformação e agregação de dados
- Padronização de respostas
- Tratamento de erros
- Documentação técnica

3. Descrição da atividade

Os alunos deverão desenvolver uma **API REST** que integre **APIs públicas** para fornecer informações combinadas sobre cidades brasileiras, incluindo dados geográficos e climáticos.

Importante: O usuário da API informará apenas o **nome da cidade**. Todo o processamento para identificar a cidade, buscar suas informações geográficas,

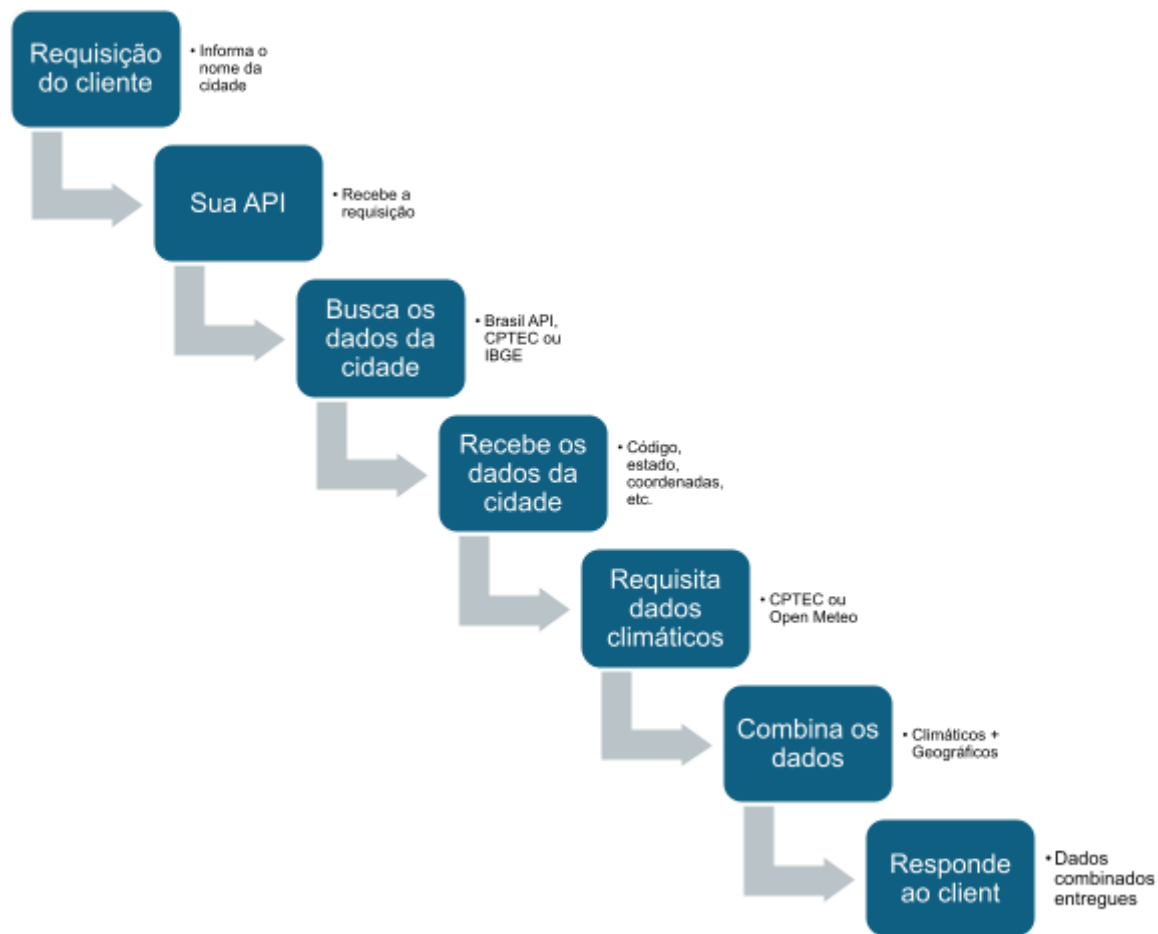
obter coordenadas e consultar dados climáticos deve ser feito pela aplicação. **Não é permitido utilizar coordenadas fixas no código.**

4. APIs sugeridas para integração:

API	Descrição	Documentação
Brasil API - IBGE	Informações sobre estados e municípios brasileiros	https://brasilapi.com.br/docs#tag/IBGE
Brasil API - CPTEC	Dados meteorológicos do CPTEC/INPE (busca por nome da cidade)	https://brasilapi.com.br/docs#tag/CPTEC
Open-Meteo	Dados climáticos gratuitos (requer latitude/longitude)	https://open-meteo.com/en/docs

Nota: Os alunos podem utilizar outras APIs públicas além das sugeridas, desde que atendam aos requisitos da atividade. O importante é que a busca seja feita pelo **nome da cidade** e que as coordenadas sejam obtidas dinamicamente.

5. Fluxo de integração esperado:



6. Especificação Técnica Detalhada

Endpoint 1: Informações da Cidade com Clima

Rota: GET /api/v1/clima/{nome_cidade}

Parâmetros de entrada:

Parâmetro	Tipo	Obrigatório	Descrição
nome_cidade	string	Sim	Nome da cidade (pode ser parcial)

Exemplo de requisição:

GET /api/v1/clima/Fortaleza

Formato de resposta (sucesso - HTTP 200):

{

```
{
  "nome": "Fortaleza",
  "estado": "CE",
  "clima": {
    "temperatura_min": 24,
    "temperatura_max": 32,
    "condicao": "Parcialmente Nublado",
    "unidades": {
      "temperatura": "°C"
    }
  },
  "consultado_em": "2025-03-15T14:30:00Z"
}
```

Nota: Os campos retornados podem variar dependendo das APIs utilizadas. O importante é que a resposta contenha informações geográficas (nome, estado) e climáticas (temperatura, condição do tempo).

Formato de resposta (erro - cidade não encontrada - HTTP 404):

```
{
  "erro": true,
  "codigo": "CIDADE_NAO_ENCONTRADA",
  "mensagem": "Nenhuma cidade encontrada com o nome informado",
  "nome_informado": "CidadeInexistente"
}
```

Formato de resposta (erro - nome inválido - HTTP 400):

```
{
  "erro": true,
  "codigo": "NOME_INVALIDO",
  "mensagem": "O nome da cidade deve conter pelo menos 2 caracteres",
  "nome_informado": "X"
}
```

Formato de resposta (erro - serviço externo indisponível - HTTP 503):

```
{
  "erro": true,
  "codigo": "SERVICO_EXTERNO_INDISPONIVEL",
  "mensagem": "Não foi possível obter dados do serviço externo. Tente novamente em alguns instantes",
  "servico": "CPTEC"
}
```

Endpoint 2: Listagem de Cidades por Estado

Rota: GET /api/v1/cidades/{sigla_uf}

Parâmetros de entrada:

Parâmetro	Tipo	Obrigatório	Descrição
sigla_uf	string	Sim	Sigla do estado (2 caracteres)

Query Parameters opcionais:

Parâmetro	Tipo	Default	Descrição
limite	integer	10	Quantidade máxima de cidades (1-100)

Exemplo de requisição:

GET /api/v1/cidades/CE?limite=5

Formato de resposta (sucesso - HTTP 200):

```
{
  "uf": "CE",
  "quantidade_retornada": 5,
  "cidades": [
    { "nome": "Abaíara" },
    { "nome": "Acarape" },
    { "nome": "Acaraú" },
    { "nome": "Acopiara" },
    { "nome": "Aiuaba" }
  ],
  "consultado_em": "2025-03-15T14:30:00Z"
}
```

Nota: Os campos retornados podem variar. O importante é retornar a lista de cidades do estado.

Formato de resposta (erro - UF não encontrada - HTTP 404):

```
{
  "erro": true,
  "codigo": "UF_NAO_ENCONTRADA",
  "mensagem": "Estado com a sigla informada não foi encontrado",
  "sigla_uf_informada": "XX"
}
```

Formato de resposta (erro - sigla UF inválida - HTTP 400):

```
{
  "erro": true,
  "codigo": "SIGLA_UF_INVALIDA",
  "mensagem": "A sigla do estado deve conter exatamente 2 letras",
  "sigla_uf_informada": "ceara"
}
```

Endpoint 3: Health Check

Rota: GET /api/v1/health

Formato de resposta (sucesso - HTTP 200):

```
{
  "status": "healthy",
  "versao": "1.0.0",
  "timestamp": "2025-03-15T14:30:00Z"
}
```

Formato de resposta (serviço degradado - HTTP 200):

```
{
  "status": "degraded",
  "versao": "1.0.0",
  "timestamp": "2025-03-15T14:30:00Z",
  "motivo": "Serviço externo indisponível"
}
```

7. Requisitos Técnicos Obrigatórios

A. Estrutura do Repositório (OBRIGATÓRIO seguir exatamente)

```
/
├── README.md           # Documentação principal
├── INTEGRANTES.md      # Dados da equipe (formato específico)
├── src/                # Código-fonte da aplicação
│   └── ...             # Organização livre
├── tests/              # Testes automatizados
│   └── ...             # Mínimo 2 testes
├── docs/
│   └── postman_collection.json # Coleção Postman exportada
```

B. Arquivo INTEGRANTES.md (formato obrigatório)

Integrantes da Equipe

```
| Nome Completo | Matrícula | Papel |
|-----|-----|-----|
| João da Silva | 2012345 | Desenvolvedor Backend |
| Maria Santos | 2012346 | Desenvolvedor Backend |
| Pedro Oliveira | 2012347 | Documentação |
```

8. Requisitos de Implementação

Requisito	Obrigatório	Descrição
Linguagem	Livre	Python, Node.js, Java, Go, etc.
Porta padrão	3000	API deve rodar na porta 3000
Formato resposta	JSON	Todas as respostas em JSON
Encoding	UTF-8	Suporte a caracteres especiais
CORS	Habilitado	Para testes via navegador

A. Testes Automatizados (mínimo 2)

Implementar pelo menos 2 testes automatizados que verifiquem:

1. Resposta correta para nome de cidade válido

2. Tratamento de erro para cidade não encontrada

9. Critérios de Avaliação

Pontuação Detalhada (Total: 5,0 pontos)

#	Critério	Pontos
1	Health check responde corretamente	0,5
2	Endpoint clima funciona (caso sucesso)	1,5
3	Endpoint clima - tratamento de erro 404	0,5
4	Endpoint clima - tratamento de erro 400	0,25
5	Endpoint cidades por estado funciona	1,0
6	Endpoint cidades - tratamento de erro 404	0,5
7	Endpoint cidades - tratamento de erro 400	0,25
8	Arquivo INTEGRANTES.md correto	0,25
9	Coleção Postman presente	0,25

10. Passo a Passo para Desenvolvimento

Etapa 1: Configuração Inicial

1. Criar repositório no GitHub
2. Configurar estrutura de pastas conforme especificado
3. Criar arquivo INTEGRANTES.md no formato exigido

Etapa 2: Implementação do Health Check

1. Criar servidor HTTP na porta 3000
2. Implementar endpoint `GET /api/v1/health`
3. Testar localmente

Etapa 3: Busca de Cidades

1. Estudar documentação das APIs (Brasil API, CPTEC, etc.)
2. Implementar busca de cidade pelo nome
3. Mapear resposta para formato esperado

Etapa 4: Integração com API de Clima

1. Estudar documentação da API de clima escolhida
2. Implementar consumo da API de clima
3. Combinar dados de localização com clima

Etapa 5: Tratamento de Erros

1. Implementar validações de entrada
2. Implementar códigos de erro padronizados
3. Testar cenários de erro

Etapas 6: Testes e Documentação

1. Escrever testes automatizados
2. Criar coleção Postman
3. Finalizar [README.md](#)

11. Orientações para Envio da Atividade

Via AVA (Obrigatório):

Todos os integrantes do grupo devem:

1. Estar cadastrados na equipe via AVA
2. Submeter um documento contendo:
 - Nome completo de todos os integrantes e matrículas
 - Link para o repositório GitHub (público)

Requisitos do Repositório GitHub:

Item	Obrigatório	Verificação
Repositório público	Sim	Script clona o repositório
Arquivo README.md	Sim	Documentação básica
Arquivo INTEGRANTES.md	Sim	Formato específico exigido
Pasta src/ com código	Sim	Código-fonte presente
Pasta tests/ com testes	Sim	Mínimo 2 arquivos de teste
Coleção Postman	Sim	Em docs/postman_collection.json

Checklist de Submissão

Antes de enviar, verifique:

- Repositório está público no GitHub

- Arquivo INTEGRANTES.md está no formato correto
- Instruções de execução estão no README
- API responde em `http://localhost:3000`
- Endpoint `/api/v1/health` retorna HTTP 200
- Endpoint `/api/v1/clima/Fortaleza` retorna dados climáticos
- Endpoint `/api/v1/cidades/CE` retorna lista de cidades
- Erros retornam os códigos HTTP corretos (400, 404, 503)
- Formato JSON das respostas segue a especificação
- Coleção Postman está em `docs/postman_collection.json`
- Existem pelo menos 2 testes na pasta `tests/`

12. Cidades para Testes

Use estas cidades para testar sua implementação:

Cidade	UF
Fortaleza	CE
São Paulo	SP
Rio de Janeiro	RJ
Brasília	DF
Salvador	BA
Belo Horizonte	MG
Curitiba	PR
Manaus	AM

13. Formação de Equipes

- **Tamanho:** 2 a 6 integrantes
- **Cadastro obrigatório:** Todos devem se cadastrar no grupo via AVA
- **Dúvidas:** Utilizar o fórum de formação de equipes

14. Pontuação Total

Categoria	Pontos
Health check	0,5
Endpoint clima (sucesso + erros)	2,25
Endpoint cidades por estado (sucesso + erros)	1,75
Estrutura e documentação	0,5
TOTAL	5,0

A atividade corresponde a **50% da nota do trimestre**.

15. Prazo para Envio

Data limite: 05/06/2026 às 23h59 - Prazo final para entrega da Atividade Parcial

Local: AVA, no local apropriado para submissão

Não serão aceitas entregas por e-mail ou após o prazo estabelecido.

16. Referências e Recursos

APIs Utilizadas

- **IBGE Localidades:** <https://servicodados.ibge.gov.br/api/docs/localidades>
- **Open-Meteo:** <https://open-meteo.com/en/docs>

Ferramentas

- **Postman:** <https://learning.postman.com/docs/>
- **Markdown:** <https://dotmd-editor.vercel.app/>

Exemplos de Código (GitHub)

- Flask API Template: <https://github.com/pallets/flask>
- Express API Template: <https://expressjs.com/>
- FastAPI Template: <https://fastapi.tiangolo.com/>

17. Suporte

Em caso de dúvidas técnicas sobre a especificação:

1. Consulte esta documentação primeiro
2. Utilize o fórum da disciplina no AVA
3. Aguarde resposta da tutoria e/ou do professor

Dúvidas sobre interpretação dos requisitos não serão respondidas individualmente por e-mail para garantir que todos os alunos tenham acesso às mesmas informações.

Bom trabalho!